Analyzing Feasibility and Effectiveness of Experts' Weight Scheduling for MoE LLMs in Memory-Constrained GPUs

Mohammed Toufikuzzaman
Dept. of CSE, PSU

Ridwanul Hasan Tanvir
Dept. of CSE, PSU

***Abstract*—Mixture-of-Experts (MoE) LLMs activate only a small subset of experts per token, so in theory they could run on smaller GPUs if we can cache and prefetch expert weights efficiently. In this project, we collect routing traces from OLMoE and GPT-OSS-20B using 500 prompts from the OpenAssistant dataset. We analyze expert usage patterns across prefill, analysis (for GPT-OSS-20B), and generation phases. We then build a simple LSTM-based predictor to predict next-token expert probabilities and test it in a lightweight expert-pool simulation. We compare LSTM-based prefetching with a standard LRU cache under different pool sizes. Our results show that LSTM helps only in a narrow range of pool sizes and does not consistently outperform LRU. We discuss why the gains are limited and outline future work with larger and more diverse traces.**

***Index Terms*—Mixture of Experts, MoE Routing, MoE Scheduling**

I. Introduction

Large Language Models have seen a meteoric rise in research and real world applications in the last few years. However, the sheer size of these models make them unsuitable to be used in consumer GPUs. In recently published LLMs [1]–[3] , including GPT-5 [4], we have seen the usage of Mixture of Experts (MoEs). In these models, after the attention layer inside each transformer layer, there are a set of smaller liners layers (identified as experts) instead of a single dense layer. A routing layer facilitates the activation if a subset of these experts during token processing. As the parameters in the dense layers constitutes a significant portion of an LLM's weight, only a fraction of total parameters are active during token processing in a MoE LLM. In a cloud environment, requests from multiple users are batched, and hence, the utilization factor of the experts remain high enough to place the whole model inside GPU VRAM. However, if the whole model is loaded in a local consumer GPU, most of the only a small fraction of the model weights are in use. From another perspective, it can be argued that if it is possible to efficiently predict the expert activation, a large MoE can be loaded in a smaller GPU, and experts' weights can be swapped in and out of main memory in the presence of high bandwidth bus. In such a scenario, we can try to efficiently prefetch the experts to reduce I/O latency. In our project, we have collected traces of experts' routing from OLMoE [2] and GPT-OSS-20B [5] models and performed statistical analysis to understand the patterns of expert activation. In particular, we have observed how the pattern changes in prefilling, thinking/analysis (chain of thought), and the generation stage. We draw the intuition that the patterns could be predictable from inexpensive computation based on the fact that router logics get saturated in the early stage of pretraining (1% data) [2]. Then we designed a simplified prediction scheme based on LSTM and simulated its performance. It should be noted that, in this course project, we did not plan to write system-level programs to test the prefetching performance in real hardware. Rather, we aimed to perform an empirical analysis of the prediction algorithm's performance. We have focused on type 2 project for this course, where our project as a whole might not have significant novel contributions, but we expected to find interesting patterns by dividing the routing traces into prefilling, analysis, and generation parts, which may guide us to propose a simple prefetching scheme. In summary, our contribution in this project is as follows.

- We have collected routing traces from two MoE models (OLMoE and GPT-OSS-20B) using a popular conversational dataset (Open Assistant).
- We have performed statistical analysis, like entropy calculation, and identified the most and least frequently used experts in different inference stages.
- We have designed an LSTM model to predict the expert activation so that experts can be prefetched well ahead of time.
- We have designed a very simple expert pool simulator to compare the LSTM's performance with the basic LRU mechanism.

II. Literature Review

Mixture-of-Experts (MoE) models increase capacity by activating only a small subset of expert feed-forward modules per token. This sparsity reduces computation but makes routing and memory management central bottlenecks, especially when expert parameters no longer fit in GPU memory. Recent work therefore focuses on two directions: (i) understanding expert activation statistics, and (ii) designing offloading, caching, and predictive scheduling strategies for memory-constrained inference.

SAFEx [6] analyzes MoE models from a safety perspective and introduces the notion of stable safety-critical experts.

By collecting large-scale routing traces, the authors identify experts whose activation strongly correlates with harmful or benign behavior and intervene at the expert level. While their goal is safety rather than speed, SAFEx demonstrates how to compute expert-level statistics such as activation frequency and stability from traces, which directly motivates our own trace-based analysis.

A second line of work targets MoE inference when experts are offloaded to CPU and fetched over PCIe. Eliseev and Mazur [7] observe that experts are frequently reused across adjacent tokens and that early-layer hidden states already indicate which experts later layers will use. They combine an LRU expert cache on the GPU with speculative prefetching based on these hidden states, overlapping expert loading with compute. Sparsity-aware expert cache systems such as MoE-Infinity [8] extend this idea by treating experts as cacheable objects backed by host memory and managing a small GPU-resident cache using usage statistics. FloE [9] further reduces transfer cost by compressing experts via channel pruning and low-bit quantization and introduces inter- and intra-expert predictors to prefetch both experts and channels. Together, these works show that exploiting expert reuse and prediction is critical for efficient offloaded MoE inference.

Other approaches modify the effective routing behavior to reduce expert usage per token. MoBiLE [10] builds a "little" expert configuration that activates only half of the default experts and runs this cheaper configuration first. A confidence test on the output determines whether to accept the token or fall back to the full "big" configuration, reusing router information to prefetch experts. Pre-gated MoE [11] changes the timing of routing decisions: block N includes a compact pre-gate MLP that predicts the expert mask for block $N+1$, allowing expert migration from CPU to GPU to overlap with computation. Both methods achieve substantial speedups with modest accuracy loss by aligning routing and communication more tightly with actual expert usage.

Across these works, expert routing is not random. It exhibits strong temporal and cross-layer regularities that can be exploited via caching, compression, and prediction. In contrast to prior system implementations, our work focuses on collecting detailed routing traces from OLMoE and GPT-OSS-20B and using them to quantify such regularities across prefill, reasoning, and generation phases. We also plan to compute the entropy of expert distributions, where high entropy indicates diverse expert usage and low entropy indicates concentrated usage. We will then use these statistics to design and simulate expert scheduling policies (e.g., LRU-style caches and simple temporal predictors) under memory constraints, providing a trace-driven perspective on the feasibility of MoE offloading strategies.

III. Experiments

A. Infrastructure Setup

As mentioned previously, our goal is to perform statistical analysis on collected MoE traces and devise a light weight scheduling plan and simulate its effectiveness. For the scope of this course project, we are not considering full scale real system integration of the scheduling method and we plan to explore that direction if the project yields promising results. Hence, our experiments require us to use a GPU for trace collection and general computing power for statistical analysis and simulation. We have set up an Azure Standard NV36ads A10 v5 instance for this purpose. This VM instance has 36 vCPU with 440 GB RAM and access to NVIDIA A10 GPU with 24GB VRAM. We have successfully tested that both OLMoE 7B with 16 bit floating point precision [2] and quantized (MXFP4) GPT-5-OSS 20B [5] models fit in this GPU for inference purpose. While setting this particular VM, we two major issues; i) A10 GPU requires NVIDIA Grid driver instead of general NVIDIA drivers as it can slice the same GPU for multiple users, ii) latest Ubuntu Kernel was not compatible with latest Grid driver and we had to do trial error with several kernel-grid versions to get it working. We include the details and step by step solution to this problem in our final code submission. In order to minimize the usage of azure credits, we are performing the statistical analysis (which can be done in CPU) in our personal machines. However, while trying to generate the trace of GPT-OSS-20B model, we identified that the quantized version crashed if hooks were attached. Hence, we temporarily used a Nvidia H100 GPU instance, to load the model in 16 bit floating point format and it solved the issue.

B. Trace Collection

We have collected MoE traces from OLMoE [2] and GPT-OSS-20B [5]. We have used the Open Assistant Dataset [12] for this purpose. It is a large-scale, crowdsourced conversational dataset developed as part of the OpenAssistant project. It contains multi-turn human–assistant dialogues. For our experiment, we have sampled 500 random first level prompts which has at least 32 characters. Then we fed these prompts to the OLMoE and GPT-OSS-20B model for inference. We configured the model to expose routing traces while performing inference. From the OLMoE model, we collec traces in two stages; i)prefill and ii) generation. Prefill is much faster and can be done in parallel, however the generation process is auto-regressive and we collected the generation traces in step by step. For our experiments the maximum number of new token that we considered was 256. Our collected trace has per token per layer information about which experts were activatasnd their activation probability. However, we have found that GPT-5-OSS does not expose the routing trace through the same Hugging Face API like other MoE models. We deisgned hooks for routing layers (similar to our Homework 3 strategy) for this model. When the quantized model is loaded, we can successfully register the hooks, but during the forward pass the model crashes. Hence, we had to load the model in a larger GPU with 16 bit floating point to get rid of this issue. It should be noted that GPT-5-OSS has an additional reasoning capability and for this model, and hence we have further divided the generated token in two categories: i)analysis (chain of thought) and ii) generation. We expected that this distinction

would reveal more fine grained expert usage patterns. During trace generation, we set the max geneation token to 512 for GPT-OSS-20B as it takes up space for analysis part. Also, we had to set the reasoning effort to low, otherwise the model would spend much of the time in analysis and fill out all the 512 allocated slots only with reasoning tokens.

C. Trace Analysis

1) Entropy: We measured routing entropy for each transformer layer. Entropy tells us how evenly the model uses its experts. We also reported an *effective number of experts* as 2^H, where H is entropy in bits. A higher value means the routing is spread across more experts.

a) OLMoE (64 experts, 16 layers).: Entropy stays high across layers. It ranges from 5.72 to 5.88 bits. This corresponds to about 52.8 to 59.1 effective experts. This suggests the model uses many experts in a fairly balanced way.

b) GPT-OSS-20B (32 experts, 24 layers).: Entropy varies more by layer. It ranges from 3.66 to 4.85 bits. This corresponds to about 12.6 to 28.8 effective experts. This suggests some layers rely on a smaller set of experts more often.

2) Heavy and Sparse Hitters: We studied which experts are used most often and which are rarely selected. This helps us understand how uneven the routing behavior is. The analysis is based on GPT-OSS-20B traces from the OpenAssistant dataset. We label the top 10% most-used experts as heavy hitters and the bottom 10% as sparse hitters.

- **Heavy hitters**: Four experts (24, 23, 11, and 10) account for about 21% of all top-1 routing decisions. Each of these experts is selected much more often than the rest. Figure 1 shows that the model relies heavily on a small subset of experts .
- **Sparse hitters**: Four experts (25, 20, 29, and 3) are selected only about 1.4-1.8% of the time. These experts are rarely used, which suggests unused capacity in the model.

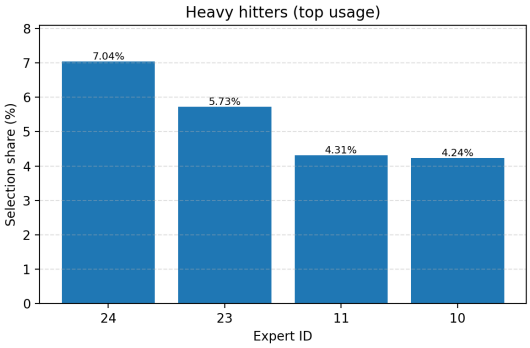

Fig. 1. Heavy hitters

3) Prefill vs. Generation Heavy-Hitter Analysis: We compared expert usage between the prefill and generation stages. We analyze the two models separately. For each stage, we count how often each expert is selected. We then identify the most-used and least-used experts.

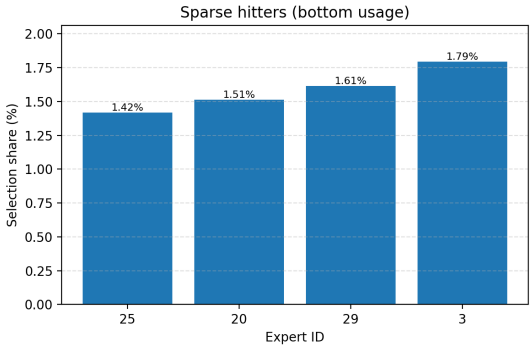

Fig. 2. Sparse hitters

In the case of GPT-OSS-20B, the most-used experts during prefill are 24, 23, 11, and 10. During generation, the most-used experts change to 31, 14, 9, and 10. Only expert 10 appears in both stages. This shows a clear shift in expert usage between stages. On the other hand, in OLMoE, the most-used experts during prefill are 4, 6, 54, 62, 1, 35, and 61. During generation, the most-used experts become 27, 21, 59, 14, 24, 61, and 33. Only expert 61 is shared between the two stages. This again shows a clear change in expert usage.

We also measured how much the overall routing distribution changes between stages. For both models, the change is noticeable. The shift is stronger in GPT-OSS-20B than in OLMoE. Both models use different experts for prefill and generation. Prefill focuses on experts that handle the input prompt. Generation relies on a different set of experts for producing new tokens. This means expert usage could be stage dependent. Comprehensive trace analysis is needed for further confirmation.

4) Heatmap: We use heatmaps to show how often each expert is selected in each layer. Each cell shows the percentage of prompt tokens that select a given expert as the top choice. This helps us see whether some experts are preferred by certain layers.

- GPT-OSS-20B has 24 layers and 32 experts in each layer.
- OLMoE has 16 layers and 64 experts in each layer.

For each layer, we count how often each expert is selected and normalize the counts.

a) GPT-OSS-20B: In GPT-OSS-20B, several layers rely on a small set of experts. In some layers, a single expert is selected for more than half of the tokens. Other layers spread traffic more evenly, but still show clear preferences. This suggests partial specialization across layers, as shown in Fig. 3.

b) OLMoE: Expert usage in OLMoE is more spread out. Most experts are selected only a small fraction of the time. A few layers show visible spikes, but they are less frequent and less strong. This suggests weaker and less consistent specialization across layers, as shown in Fig. 4.

Overall, GPT-OSS-20B shows stronger expert preferences across layers. OLMoE uses a wider set of experts with fewer strong patterns.

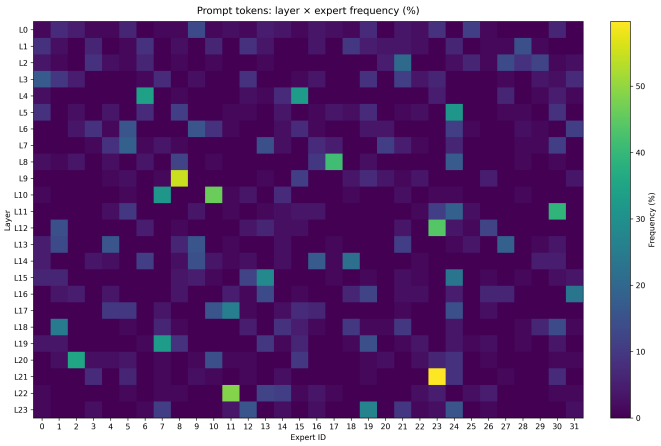

Fig. 3. Layer-wise expert selection frequency for GPT-OSS-20B

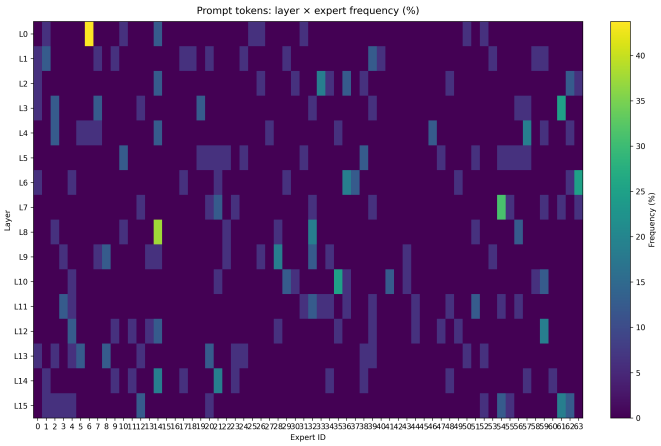

Fig. 4. Layer-wise expert selection frequency for OLMoE

D. Expert Prediction

We designed an LSTM-based expert prediction model using the routing traces. Our routing traces contained per token per layer activation data of top-K experts. Our prediction pipeline was quite simple. One of the assumptions that we made was that, while a single token is being processed it would be best if we could already have all its required experts in GPU memory. Besides, while this one is being processed, we should try to prefetch the next tokens potential expert. A good prediction would give us improved hit rate in expert pool (a set aside memory for experts' weights) and that would ultimately help run a large model in a small GPU with little to no performance penalty. The current expert prediction methods has already been discussed in Section II. Some of these methods rely on real time router activation and other activation values from the model and some others depends on rigorous system level implementation. However, we had limited access to GPU and hence only could design an offline model by using trace data only and we our project's scope excluded complex real system level implementation. In this section, we describe the basic design of our LSTM model, how it was trained and how did it perform on the test set.

TABLE I
DATASET SIZE AND STRUCTURE.

Attribute	Value
Training files	400
Test files	100
Training sequences	98,400
Test sequences	24,600
Input sequence length	10
X_{train} shape	(98,400, 10, H, W)
Y_{train} shape	(98,400, H, W)
X_{test} shape	(24,600, 10, H, W)
Y_{test} shape	(24,600, H, W)

1) Dataset: We used 80% of the trace data for training and rest for testing the LSTM model. Each sample had two parts, the prefill and the generation trace. The prefill traces are generated from the user prompts and hence they have vairbale number of tokens. On the other hand we have max length of generation which is 256 for OLMoE and 512 for GPT-OSS-20B. We processed the data so that it looks more like a time series prediction. We set a context window size of 10. We converted the traces in such a way that for each sample, the data point is the router activation pattern in all layers for last 10 tokens and label is the activation pattern for current token. This essentially generated lots of training and test samples from the traces of 500 prompts. A summary of the dataset is given in Table I. The sizes for H and W varies for OLMoE and GPT-OSS-20B.

2) Model Architecture: We used a BiLSTM-based model to predict expert activation patterns from the routing traces. The model was designed to be configurable so that we could use the same architecture for both OLMoE and GPT-OSS-20B. The only parameters that change across models are the number of layers and the number of experts per layer.

The input to the model is a probability distribution tensor of shape $[B, 10, L, E]$, where B is the batch size, 10 is the context window size, L is the number of transformer layers, and E is the number of experts per layer. Each time step contains the router probabilities for all experts in all layers. Since an LSTM expects a sequence of vectors, we flattened the last two dimensions and converted the input into $[B, 10, L \times E]$. Therefore, the effective input feature size is $L \times E$ and it changes depending on the model configuration.

We then passed this sequence through a two-layer bidirectional LSTM with hidden size 512. Because it is bidirectional, the hidden states from both directions are concatenated, resulting in a $2 \times 512 = 1024$ dimensional representation per time step. We also used dropout of 0.2 between the LSTM layers to reduce overfitting.

After the BiLSTM, we applied a simple attention mechanism to compute a weighted summary of the 10 time steps. The attention layer is a linear projection from 1024 to 1, producing one attention score per time step. We normalized these scores using softmax across the time dimension and computed a weighted sum of the BiLSTM outputs to obtain a single context vector of shape $[B, 1024]$.

Finally, we projected this context vector back to the expert space using a fully connected layer from 1024 to $L \times E$. We reshaped the output to $[B, L, E]$ and applied a softmax over the expert dimension so that each layer outputs a valid probability distribution over experts. In other words, for each layer, the model predicts which experts are likely to be activated for the next token. We trained the BiLSTM model using a batch size of 32 for 50 epochs. The model parameters were optimized using a learning rate of 1×10^{-4}. Mean squared error (MSE) was used as the loss function to measure the difference between the predicted and ground-truth expert activation probability distributions.

E. Simulation

In order to evaluate the performance of our LSTM-based expert prediction model, we designed a simple simulation that mimics an expert pool in GPU memory. The simulation was parameterized the number of layers, numbear of experts per layer, and router's top-K value.

1) Setup: We assume a Mixture-of-Experts model with L layers and E experts per layer, resulting in $L \times E$ total experts. For each token, the router selects the top-K experts per layer, and therefore $L \times K$ experts are required to process a single token. All values of L, E, and K are model-specific and are set according to the target model. We simulated an expert pool that represents a fixed-capacity GPU memory reserved for storing expert weights. The pool size was defined as a fraction of the total number of experts. We set the pool size to be a fraction of total number of experts in the model and the fraction was varied in the range of 20% to 80% in our experiments. At the beginning of the simulation, the pool was empty.

For each token, we obtained the ground-truth set of required experts from the collected traces and checked whether each required expert was already present in the pool. Experts present in the pool were counted as hits, while missing experts were counted as misses and needed to be loaded into the pool.

2) LRU Baseline: As a baseline, we implemented a standard least-recently-used (LRU) policy. When a token was processed, all required experts were checked against the pool. Experts found in the pool were marked as recently used. Missing experts were inserted into the pool, and if the pool capacity was exceeded, experts were evicted according to the LRU rule until the pool fit within its size limit. This baseline captures cache-like behavior without any prediction and previous works on expert predcition also used this method as a baseline.

3) LSTM-Based Prefetching: We then augmented the LRU policy with LSTM-based prefetching. Using the routing traces, we maintained a context window of the previous 10 tokens. Before processing the next token, we used the trained BiLSTM model to predict the expert activation probability distribution for the upcoming token. From this prediction, we selected the top-K experts per layer and attempted to prefetch them into the pool.

If the pool had additional free capacity after inserting the first predicted expert set, we performed additional prediction steps. In these steps, the context window was shifted, and the previous prediction was treated as part of the updated context. This process continued until the pool was filled or no further predictions were made.

When a token was processed and a miss occurred, the required experts were first loaded into the pool. Immediately after this update, we ran the predictor again using the updated context to prefetch experts for the next token. Overall, this approach can be viewed as an LRU policy augmented with LSTM-based expert prefetching.

4) Metrics: We measured hit rate and processing time for both the LRU baseline and the LSTM-based simulation. Hit rate was defined as the fraction of required experts that were already present in the pool when a token was processed. We reported hit rates both overall and separately for different stages of execution, including prefill and generation. As GPT-OSS-20B exposes analysis traces, we additionally reported metrics for analysis tokens, whereas for OLMoE, we reported only prefill and generation results.

5) Simulation Results: Table II summarizes the main observation. In OLMoE, LSTM-based prefetching helps mostly in the mid range pool sizes. For example at ∼40%–50% pool, we got around 4.5% absolute improvement in overall hit rate compared to LRU. But when the pool is too small (around 20%) or too large (60%+), LSTM did not help and sometimes hurt. One reason is, when pool is tiny, wrong prefetch may evict useful experts too often. And when pool is already large, LRU is already very strong so there is not much space to improve.

For GPT-OSS-20B, LRU already gives very high hit rates as the pool grows, and our LSTM prefetching was mostly similar or slightly worse in most settings. We did see almost no difference at large pools (70%–80%), where both policies are near saturated. We also compared the average simulation time between the two policies. On average, the LRU-only simulation took around 0.025/0.04 (OLMOE/GPT-OSS-20B) seconds per run, since it only involves simple cache bookkeeping. In contrast, the LSTM+LRU simulation took roughly 1.2/2.4 (OLMOE/GPT-OSS-20B) seconds per run, mainly due to repeated LSTM forward passes during prediction and pool filling. This gap is expected given our Python-based offline setup. However, it highlights that any practical benefit from prediction must be large enough to justify this additional overhead, especially in real systems where latency is critical.

Figures 5 and 7 show the overall hit rate as a function of expert pool size for OLMoE and GPT-OSS-20B, respectively. Figure 6 breaks down the results by stage. Generation tokens consistently achieve higher hit rates than prefill tokens. The LSTM-based approach provides some improvement during generation, but its effect on prefill remains marginal. This suggests that expert activation patterns during prefill are harder to predict and may be too diverse for a short-context LSTM to model accurately. On the other hand, Figure 8 shows stage-wise behavior for GPT-OSS-20B. The analysis stage exhibits lower hit rates compared to generation, but the difference shrinks with larger pool sizes. Across all stages, the LSTM

TABLE II
SIMULATION RESULTS (OVERALL HIT RATE) ACROSS POOL SIZES.

Model	Pool	LRU	LSTM+LRU	Δ
OLMoE	20%	0.377	0.242	-0.135
OLMoE	30%	0.542	0.545	+0.003
OLMoE	40%	0.606	0.654	+0.048
OLMoE	50%	0.668	0.713	+0.045
OLMoE	60%	0.776	0.758	-0.018
OLMoE	70%	0.884	0.854	-0.030
OLMoE	80%	0.953	0.932	-0.021
GPT-OSS-20B	20%	0.505	0.370	-0.136
GPT-OSS-20B	30%	0.679	0.666	-0.013
GPT-OSS-20B	40%	0.797	0.781	-0.016
GPT-OSS-20B	50%	0.888	0.880	-0.008
GPT-OSS-20B	60%	0.942	0.938	-0.004
GPT-OSS-20B	70%	0.974	0.974	-0.000
GPT-OSS-20B	80%	0.985	0.985	+0.001

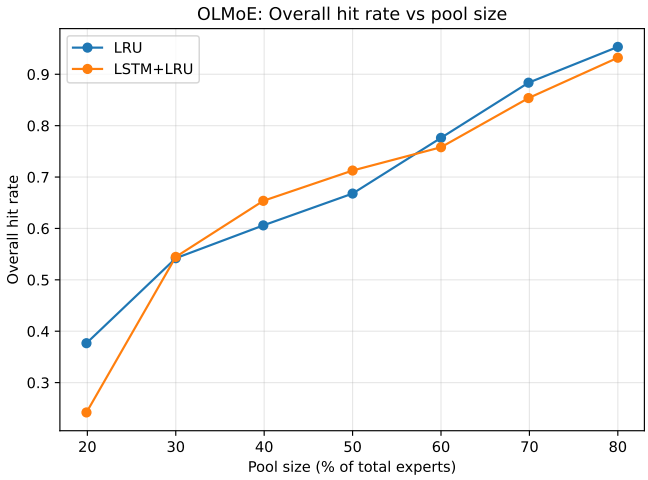

Fig. 5. OLMoE overall hit rate versus expert pool size.

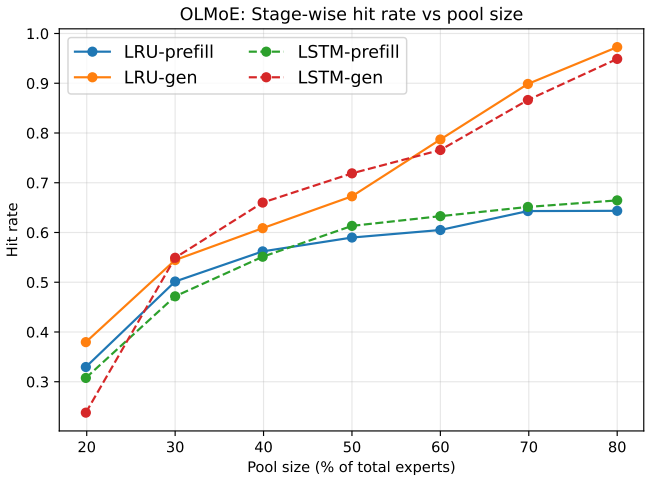

Fig. 6. OLMoE stage-wise hit rate versus expert pool size.

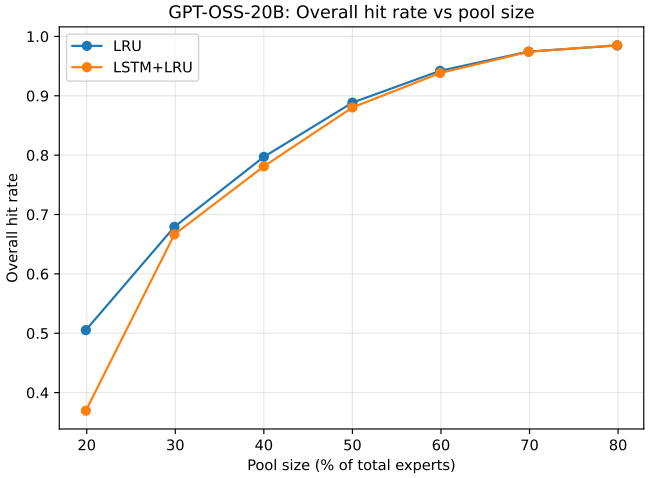

Fig. 7. GPT-OSS-20B overall hit rate versus expert pool size.

predictor provides little to no improvement. Also, no specific phase dominate the other.

Unfortunately, these results indicate that while LSTM-based expert prediction can help in a narrow range of memory configurations, it is not a universally effective replacement for simple caching policies.

IV. DISCUSSIONS

A. Feasibility of Expert Prediction

Prior studies have shown that expert routing in MoE models is not random and exhibits temporal and cross-layer correlations that can be exploited for prediction and prefetching [7]–[9]. These works suggest that a subset of experts tend to co-activate across nearby tokens or layers, making speculative loading feasible in practice.

In our experiments, however, we did not observe strong or consistent evidence of such correlations translating into reliable prediction gains. Though, the heatmap analysis showed some sort of expert preference, both the LSTM-based predictor and the LRU baseline showed slight or no improvement, with LSTM helping only in a narrow range of intermediate pool sizes. Technically, the heatmap spikes indicate LRU without any prediction could be useful as some experts show higher usage patterns. But we were only able to gain a good hit rate at higher expert pool capacity. One possible reason is the relatively small scale of our trace dataset where test samples did not exactly represent the training samples. We collected routing traces from only 500 prompts, which was constrained by limited access to cloud GPUs and the high cost of large-scale inference. This dataset size may be insufficient to expose stable long-range routing patterns, especially for complex models such as GPT-OSS-20B.

Another important factor is the treatment of the reasoning (analysis) phase in GPT-OSS-20B. Although we expected the chain-of-thought stage to reveal some reuse patterns, practical constraints limited our setup. We had to limit the maximum generation length to 512 tokens and reduced the reasoning effort to low to prevent analysis tokens from hogging the output. As a result, many samples contained only minimal

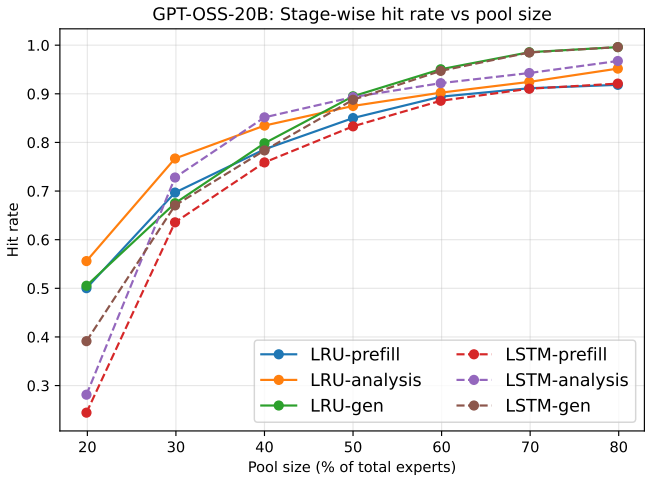

Fig. 8. GPT-OSS-20B stage-wise hit rate versus expert pool size.

analysis content, often consisting of short generic reasoning such as "provide answer." Since we used a benign conversational dataset, this further reduced the diversity of reasoning traces. With a diverse dataset and lager trace collection, expret prediction could be feasible.

B. Future Works

There are several directions for extending this work. First, we plan to collect routing traces from a more diverse dataset with longer generation lengths and higher reasoning effort, especially for GPT-OSS-20B. This would allow us to better study expert behavior during the chain-of-thought phase.

Second, given the computational cost of LSTM-based prediction, we plan to explore cheaper alternatives. One of the ideas that we thought but could not complete during this project's scope is to perform layer-wise covariance and correlation analysis of expert activations. If strong statistical dependencies are observed, we will design a lightweight, phase-aware (prefill, reasoning, and generation) Markov chain model. Such a predictor could offer faster inference-time prediction with lower overhead.

C. AI Usage Disclosure

We used common AI-assisted development tools as part of our implementation workflow. Specifically, we used IDE's AI features for code cleanup, formatting, verbose logging, and debugging. In some cases, we used autocomplete for very basic tasks, such as file handling or boilerplate code generation.

Our primary tools included VS Code Copilot and Anthropic's Claude Code. All AI-assisted outputs were carefully reviewed, tested, and validated by the us.

V. Conclusions

We collected routing traces from OLMoE and GPT-OSS-20B and studied expert usage across prefill, analysis, and generation phases. We trained a simple LSTM predictor and compared it with an LRU cache in an expert-pool simulation. Overall, LSTM helped only in a small range of pool sizes and did not consistently beat LRU. In future, we plan to use larger and more diverse traces and test cheaper predictors like correlation analysis and phase-aware Markov models.

References

[1] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[2] N. Muennighoff, L. Soldaini, D. Groeneveld, K. Lo, J. Morrison, S. Min, W. Shi, P. Walsh, O. Tafjord, N. Lambert *et al.*, "Olmoe: Open mixture-of-experts language models," *arXiv preprint arXiv:2409.02060*, 2024.

[3] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[4] OpenAI, "Gpt-5 system card," 2025, accessed: 2025-11-16. [Online]. Available: https://openai.com/index/gpt-5-system-card/

[5] ——, "Gpt-5 oss system card," 2025, accessed: 2025-11-16. [Online]. Available: https://openai.com/index/gpt-oss-model-card/

[6] Z. Lai, M. Liao, B. Wu, D. Xu, Z. Zhao, Z. Yuan, C. Fan, and J. Li, "Safex: Analyzing vulnerabilities of moe-based llms via stable safety-critical expert identification," *arXiv preprint arXiv:2506.17368*, 2025.

[7] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.

[8] L. Xue, Y. Fu, Z. Lu, C. Sun, L. Mai, and M. K. Marina, "Moe-infinity: Efficient moe inference on personal machines with sparsity-aware expert cache," 2025.

[9] Y. Zhou, Z. Li, J. Zhang, J. Wang, Y. Wang, Z. Xie, K. Chen, and L. Shou, "Floe: On-the-fly moe inference on memory-constrained gpu," *arXiv preprint arXiv:2505.05950*, 2025.

[10] Y. Zhao, Y. Qin, Y. Wang, X. Yang, H. Han, S. Wei, Y. Hu, and S. Yin, "Mobile: Efficient mixture-of-experts inference on consumer gpu with mixture of big little experts," *arXiv preprint arXiv:2510.12357*, 2025.

[11] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, "Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 1018–1031.

[12] A. Köpf, Y. Kilcher, D. Von Rütte, S. Anagnostidis, Z. R. Tam, K. Stevens, A. Barhoum, D. Nguyen, O. Stanley, R. Nagyfi *et al.*, "Openassistant conversations-democratizing large language model alignment," *Advances in neural information processing systems*, vol. 36, pp. 47 669–47 681, 2023.